implementing and analyzing td algorithms

elsayed elmandoua - 20596379 - CISC 856 - Reinforcement Learning - Queen's University - June 2026

0. Overview

This report presents the implementation and analysis of two fundamental temporal-difference (TD) control algorithms, SARSA (on-policy) and Q-learning (off-policy), in a 10×9 gridworld environment with walls, a start state (S), and a goal state (G). The agent navigates the grid by choosing one of four actions (up, right, down, left) at each step. Bumping into a wall incurs a −5 penalty and keeps the agent in place; reaching the goal yields +10 and resets to the start. The discount factor is γ = 0.9.

The assignment consisted of four analysis tasks:

1. Plot the average reward as a function of steps for both algorithms at ε = 0.1.
2. Repeat the above plot for ε = 0.1, 0.5, and 1.0.
3. Describe how different values of ε affect training in both algorithms.
4. Determine the optimal value of ε.

1. Algorithms

1.1 SARSA (On-Policy)

```
Q(S, A) ← Q(S, A) + α [R + γ Q(S', A') − Q(S, A)]
```

The next action A' is picked using the same ε-greedy policy. Since behavior and target are the same policy, SARSA learns the value of whatever policy its actually running — random moves and all.

1.2 Q-Learning (Off-Policy)

```
Q(S, A) ← Q(S, A) + α [R + γ max_a' Q(S', a') − Q(S, A)]
```

Here behavior is ε-greedy but the target is greedy (max Q). This is the key difference: Q-learning can converge to Q* while exploring randomly.

2. Setup

Parameter	Value
Learning rate (α)	0.1
Discount factor (γ)	0.9
Default ε	0.1
Steps per run	100,000
Runs per condition	5
Epsilons tested	0.1, 0.5, 1.0

Averaged over 5 runs, with generators seeded. Rewards smoothed with a 500-step moving average.

3. Results

3.1 Task 1 - Average Reward at ε = 0.1

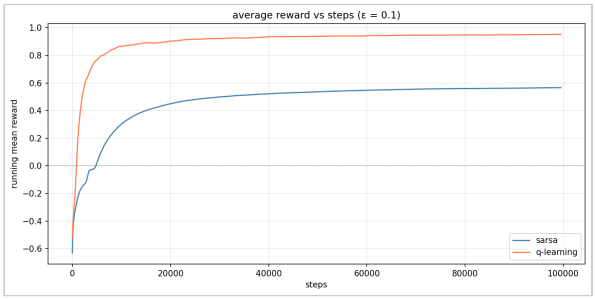


Figure 1: Running mean reward over 100k steps. SARSA in blue, Q-learning in coral. Smoothed with 500-step window.

SARSA got a mean reward of **0.6373** and Q-learning got **0.9384**. So Q-learning did better overall. The reason is the max operator — even when the agent is exploring, the update still chases the best possible action. SARSA learns whatever the agent actually ends up doing, random moves included, so it settles lower.

Both curves shoot up in the first ~10,000 steps as the agents figure out where the goal is, then level off. The wiggles after that are from the 10% random exploration still going on. The gap between the two lines shows Q-learning pulling ahead over time as its off-policy updates accumulate.

3.2 Task 2 - Effect of Epsilon

Figure 2: Reward vs steps for SARSA (left) and Q-learning (right) at ε = 0.1, 0.5, 1.0. Averaged over 5 seeds.

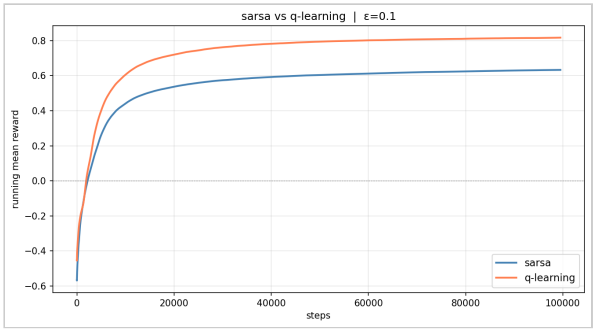
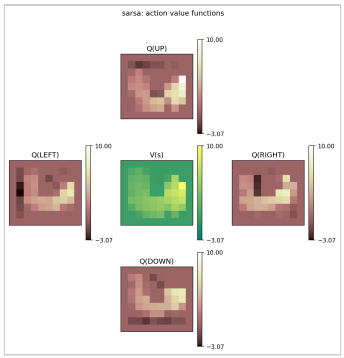
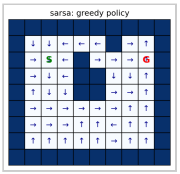
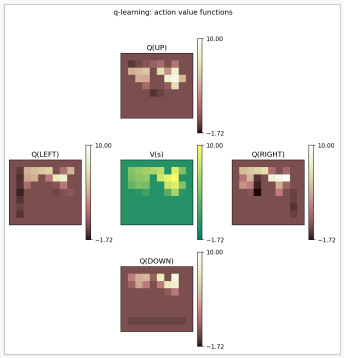


Figure 3: Direct comparison of SARSA and Q-learning at ε = 0.1.

3.3 Learned Policies

Both algorithms at ε = 0.1:

Algorithm	Action Values	Greedy Policy
SARSA		

Algorithm	Action Values	Greedy Policy
		
Q-learning		

Both algorithms found decent paths from start to goal going around the walls. The Q-values are highest near the goal and drop off as you move away, which is exactly what TD learning should produce.

---

4. Analysis

4.1 Task 3 - How Epsilon Affects Training

When ε = 0.1, both algorithms converge fast and reach the highest average reward. 90% of the time the agent follows what it knows, so it avoids walls and finds the goal consistently. Q-learning has a slight edge here — the max operator means it always targets the best action even when exploring. SARSA trails a bit because its update includes whatever random action was taken next, so the noise bleeds into the values.

When ε = 0.5, half the moves are random. Learning slows down a lot and the agent hits way more walls. SARSA suffers more because each update bakes in the random action that just happened — the noise goes straight into the Q-values. Q-learning resists better since max Q filters out the random action and still tracks the best option.

When ε = 1.0, no learning occurs, and because there are wall penalties and goal rewards, the average reward is near zero and the Q-table is just noise.

More exploration should be good in theory, but in this case will result mainly in more wall penalties. SARSA suffers more from wall penalties since it learns based on a policy that includes all random actions, while Q-learning learns based on policies that do not include the actual outcomes of the random actions.

4.2 Task 4 - Optimal Epsilon

For both algorithms **ε = 0.1** is optimal.

10% randomness is enough to give the best chance of finding the goal quickly (all Q's = 0), while 90% exploitation minimizes wall penalties and maximizes goal rewards.

Why not lower? ε = 0 (all Q = 0) will cause the agent to always pick UP and never attempt any other action, eventually getting stuck in an infinite UP loop.

Why not higher? Increasing ε after the best path is discovered would result in extra wall penalties for both algorithms and is not beneficial.

While Q-learning does tolerate more noise than SARSA since Q-learning uses max Q for all possible next actions, best trade-off for both algorithms is **ε = 0.1**.

References

Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3–4), 279–292.